```
template<class E> class MyList {
  class Node {
    friend class MyList<E>;
    E data;
    Node* next = nullptr;
  }; // end of class Node
  Node* head = new Node;
public:
  MyList() = default;
  MyList(const MyList&) = delete;
  void add(const E& data) {
    Node* node = new Node;
    node->data = data;
    node->next = head->next;
    head->next = node;
  }
  ~MyList() {
    Node *node = head, *next = head->next;
    while(node != nullptr) {
        next = node->next;
        delete node;
        node = next;
    }
  }
  template<class Function>
  void applyOnList(Function func)const {
    Node* node = head->next;
    while(node != nullptr) {
        func(node->data);
        node = node->next;
    }
  }
};
```

<u>נתון</u>: הקוד תקין וניתן לכתוב עבורו main תקין שיעשה שימוש בכל הפונקציות שהוגדרו.

בעמוד הבא מוצגים קטעי main כולל קוד עזר שעושים שימוש בקוד שלמעלה (כל main עומד בפני עצמו).

```
// M A I N - 1
int main() {
  MyList<int> m1;
  m1.add(9);
  m1.add(8);
  // m1.applyOnList( [](auto& val){ val *= 2;} );
  m1.applyOnList( [](const auto& val){cout << val << endl;} );
}
```

```
// M A I N - 2 (+ 2 additional structs)
struct A {
    friend ostream& operator<<(ostream& out, const A& a) {
        return out << "A";
    }
    virtual ~A() { cout << "~A" << endl; }
};

struct B : public A {
    friend ostream& operator<<(ostream& out, const B& b) {
        return out << "B";
    }
    virtual ~B() { cout << "~B" << endl; }
};

int main() {
  MyList<A> m2;
  m2.add(A());
  m2.add(B());
  m2.applyOnList( [](const auto& val){ cout << val << endl;} );
}
```

```
// M A I N - 3
int main() {
  MyList<std::string> m3;
  m3.add("Hello");
  m3.add("World");
  MyList<MyList<std::string>> m4;
  m4.add(m3);
  m4.add(m3);
  m4.applyOnList(
    [](const auto& list){
        list.applyOnList(
          [](const auto& val){
              cout << val << endl;
        });
  });
}
```

קטע קוד B

```
 1. template<class E> class MyList {
 2.   class Node {
 3.     friend class MyList<E>;
 4.     E data;
 5.     Node* next = nullptr;
 6.     ~Node() {delete next;}
 7.   }; // end of class Node
 8.   Node* head = new Node;
 9. public:
10.   MyList() = default;
11.   MyList(const MyList&) = delete;
12.   void add(const E& data) {
13.     Node* node = new Node;
14.     node->data = data;
15.     node->next = head->next;
16.     head->next = node;
17.   }
18.   ~MyList() {  delete head;  }
19.   class iterator {
20.       friend class MyList;
21.       Node* node;
22.     public:
23.       E& operator*() {
24.           return node->data;
25.       }
26.       bool operator!=(iterator other) {
27.           return other.node != node;
28.       }
29.       iterator operator++() {
30.           node = node->next;
31.           return *this;
32.       }
33.   };
34.   iterator begin() {
35.       iterator i;
36.       i.node = head->next;
37.       return i;
38.   }
39.   iterator end() {
40.       iterator i;
41.       i.node = nullptr;
42.       return i;
43.   }
44. };
        // continues next page
```

```
45. int main() {
46.     MyList<int> m1;
47.     m1.add(9);
48.     m1.add(8);
49.     for(auto val : m1) {
50.         cout << val << endl;
51.     }
52.  }
```

קוד C – לשאלות רשות

```
template<class E> class MyList {
  class Node {
    friend class MyList<E>;
    E data;
    Node* next = nullptr;
    ~Node() { delete next; }
  };
  Node* head = new Node;
public:
  MyList() = default;
  MyList(const MyList& list) {
      list.applyOnList([this](auto val) { add(val); });
  }
  MyList& operator=(MyList list) {
      std::swap(head, list.head);
      return *this;
  }
  MyList(MyList&& list) {
      std::swap(head, list.head);
  }
  void add(const E& data) {
    Node* node = new Node;
    node->data = data;
    node->next = head->next;
    head->next = node;
  }
  ~MyList() { delete head; }
  template<class Function>
  void applyOnList(Function func)const {
    Node* node = head->next;
    while(node != nullptr) {
        func(node->data);
        node = node->next;
    }
  }
};
```

// continues next page

```
int main() {
    MyList<std::string> m3;
    m3.add("Hello");
    m3.add("World");
    MyList<MyList<std::string>> m4;
    m4.add(std::move(m3));
    m4.add(std::move(m3));
    m4.applyOnList(
        [](const auto& list){
            list.applyOnList(
                [](const auto& val){
                    cout << val << endl;
                });
        }
    );
}
```

בהצלחה !